

Nakijk 01_03 — V4

NAKIJK V4 · Hoofdstuk 01 · Twee routeparameters in één route · bij `Oefening_01_03.md` · vierde versie van je code, eindronde, 4 oktober 2026

Opmerking

Dit is een nakijk van jouw code, geen oplossing. Er staat geen verbeterde code in.

Dit is **V4**, de laatste nakijk van deze oefening. De punten uit deel B en C van V3 laat je bewust staan. Ze komen hieronder alleen nog terug als overzicht.

In één oogopslag

Je code is opnieuw getest met alle tien de voorbeelden uit de opgave, met `/api` ervoor. **Alle tien** geven letter voor letter het gevraagde antwoord. Er blijft geen enkele fout over die een verkeerd antwoord geeft.

#	Fout uit V1 tot V3	Status in V4
1	namen in de socketroute ≠ namen van de parameters	opgelost
2	namen in de voedingsroute ≠ namen van de parameters	opgelost in V4
3	slotenroute begon met <code>/</code> , en de namen verschilden	opgelost
4	montage had de verkeerde socket	opgelost
5	sloten en modules in de omgekeerde volgorde doorgegeven	opgelost
6	de naam van de build stond vooraan in het antwoord	opgelost

Voorbeelden uit de opgave	Juist
punt 2, socket	2 van 2
punt 3, voeding	3 van 3
punt 4, sloten	3 van 3
punt 5, build	2 van 2

A. Wat beter is dan in V3

- **Fout 2 is opgelost.** De namen tussen de accolades in de voedingsroute en de parameters van `VoedingAdvies` zijn nu dezelfde. Daardoor komen de getallen uit de URL weer aan, en geven de drie voedingsvoorbeelden elk hun eigen, juiste antwoord.
- Je hebt het attribuut en de kop van de methode deze keer samen aangepast. Dat is precies de gewoonte die fout 2 voorkomt.
- De hulpmethode heet niet meer `BerekenWattage`. `MargeVoeding` sluit beter aan bij wat ze doet: ze kijkt naar de marge tussen voeding en verbruik.

B. Eén nieuw opkuispunt, zonder invloed op het antwoord

Dit punt is nieuw in V4. Het is geen fout, en je API antwoordt er niet anders door. Je mag het dus laten staan, net als de punten uit V3 in deel C.

- De parameters van `MargeVoeding` heten `BerekenWattage` en `BerekenGebruik`. Twee dingen vallen op. Ze beginnen met een hoofdletter, terwijl al je andere parameters met een kleine letter beginnen, zoals de afspraak in C# is. En ze beginnen met een werkwoord, waardoor `BerekenWattage - BerekenGebruik` leest alsof je twee methodes oproept. De oude naam van je hulpmethode is als het ware in de parameter terechtgekomen.

C. Wat je bewust laat staan

Punt uit V3	Status
route begint met <code>api/</code> , de opgave toont de voorbeelden zonder	bewust behouden
bij één vrij slot zegt je API <code>slot</code> in plaats van <code>sloten</code>	bewust behouden
de constante <code>Marge</code> staat tussen twee endpoints in	bewust behouden
lege regel tussen attribuut en methode	bewust behouden

Geen van deze punten verandert een antwoord uit de opgave. Hou alleen in je achterhoofd dat bij een examen de letterlijke tekst uit de opgave telt, zoals bij `sloten`.

D. Wat je uit deze oefening meeneemt

Over vier versies kwamen telkens dezelfde vier mechanismen terug. Ze gelden voor elke controller die je nog schrijft.

Mechanisme	Waar je het tegenkwam
Een naam tussen accolades in de route koppelt alleen aan een parameter met exact dezelfde naam. Hernoem altijd aan beide kanten tegelijk.	fout 1, 2 en 3
Een <code>int</code> die nergens gevonden wordt, krijgt stil de waarde 0, zonder foutmelding. Hetzelfde antwoord bij elke invoer is daarvan het teken.	fout 2, in V1 én in V3
Een methode-oproep koppelt de argumenten op volgorde, niet op naam.	fout 5
Een route die met <code>/</code> begint, negeert de route van de controller.	fout 3

E. Afsluiten

Je antwoorden kloppen. Leg je endpoints als laatste stap nog naast de nakijkpunten in `Oplossing_01_03.md`. Daar zie je ook hoe de oplossing met dezelfde opgave omgaat, en of je ergens een andere, even goede keuze maakte.